

Python Print Format

Back-to-Basics Series

Print Format

There are several ways to present the output in a human-readable form, or written to a file for future use. We will only focus on the following two for the purpose of this course:

- 1. Using `print()` Function**
- 2. Using `string format()` method**
3. Using `String` method
4. Using `String` Literals (Python 3.6)
5. Using `String` modulo operator(`%`)

1. Using `print()` function

Syntax for print function:

```
print(value1, value2, ..., sep='█',  
end='\n', file=sys.stdout, flush=True)
```

Examples

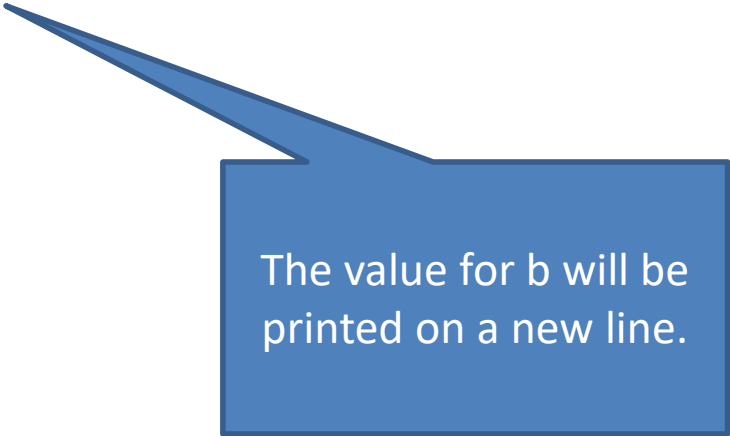
```
>>> value = 3.564
```

```
>>> print("a = ", value)
```

```
>>> print("b = ", value+10)
```

```
a = 3.564
```

```
b = 13.564
```



The value for b will be printed on a new line.

Examples

```
>>> print("a", "b")
```

There will be a spacing
between "a" and "b".

```
a b
```

```
>>> print("a", "b", sep="")
```

```
ab
```

```
>>> print(192, 168, 178, 42, sep=".")
```

```
192.168.178.42
```

```
>>> print("a", "b", sep=":-)")
```

```
a:-)b
```

Examples

By default, a print call will always end with a newline.

```
>>> for i in range(4):  
        print(i)
```

0

1

2

3

Examples

To change this behaviour, we can assign an arbitrary string to the keyword *end* parameter.

```
>>> for i in range(4):  
        print(i, end=" ")
```

0 1 2 3

```
>>> for i in range(4):  
        print(i, end=" -> ")
```

0 -> 1 -> 2 -> 3 ->

Examples

By default, the output of the print function is send to the standard output stream (sys.stdout).

```
>>> from sys import *  
  
>>> print("Print output to screen.",  
file=stdout)
```

Print output to screen.

Examples

We can send the output to a file instead.

```
>>> f = open("data.txt", "w")  
  
>>> print("Print output to file.", file=f)  
  
>>> f.close()
```

A text file `data.txt` will be created.

2. Using string `format()` method

The `format()` method inserts the specific value(s) inside the string's placeholder defined by the curly brackets and returns the formatted string.

```
print('{}', {} ... '.format(value1, value2...))
```

We can include formatting type within the placeholders.

Examples

1. Specifying the variables and the width of the string within the placeholders:

`'{var1:width1}{var2:width2}...' .format(value1, value2)`

```
names = ['Albert', 'Bob', 'Chloe', 'Desmond', 'Eve']

print('{0:8}{1:15}'.format('Index', 'Name'))
i = 1
for ele in names:
    print('{0:8}{1:15}'.format(str(i), ele))
    i += 1
```

By default, string will be aligned left, and integer/float will be aligned right.

Index	Name
1	Albert
2	Bob
3	Chloe
4	Desmond
5	Eve

Try:

`{0:<8}` to align left

`{0:>8}` to align right

`{0:^8}` to align center

Examples

2. Specifying the decimal places for float within the placeholders:

```
'{var1:width1.2f}'.format(float1)
```

```
1 prices = [('shirt', 12), ('pen', 1.5), ('cake', 4.56789)]
2
3 print('{0:8}{1:10}{2:>6}'.format('Index', 'Item', 'Price'))
4 i = 1
5 for item, price in prices:
6     print('{0:8}{1:10}{2:6.2f}'.format(str(i), item, price))
7     i += 1
```

Index	Item	Price
1	shirt	12.00
2	pen	1.50
3	cake	4.57

The End
